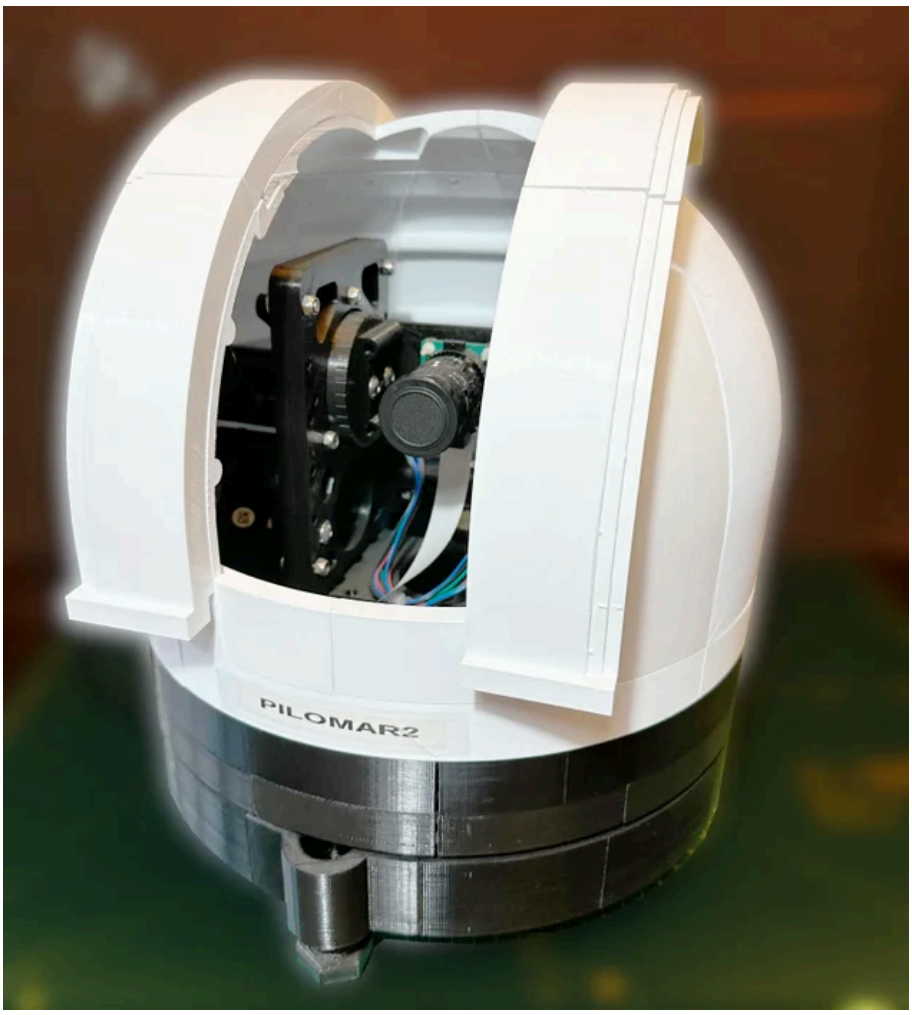
Pi-lomar - 3D Printed Working Miniature Observatory Using Raspberry Pi

By MattHh in CircuitsRaspberry Pi
Published Nov 13th, 2023
(cc) BY-NC-SA

Introduction: Pi-lomar - 3D Printed Working Miniature Observatory Using Raspberry Pi







Always under development

This is a working miniature observatory. It uses a Raspberry Pi computer for the brains, a Raspberry Pi RP2040 based microcontroller to control the movement, and the Raspberry Pi Hi Quality camera as the telescope. The Raspberry Pi operates in 'headless' mode, so you can operate it remotely from the comfort of your home if you have a home network. You can select targets from a large database and Pilomar will find and track the target if set up correctly. It will then start taking photographs which you can download and 'stack' to create more detailed images.

When Raspberry Pi launched the Hi Quality camera solution in 2020 I wondered if it was possible to use it for astrophotography. What followed was a 2 year hobby project which taught me some basics about stepper motors, microcontrollers, simple electronics, CAD development and more. I shared an image of the first version on social media and it drew some interest, now after a couple of winters of using and refining the design... here is an outline of how I made it. I am not an expert in any single element of this, everything can be better, but it works, and if I can make one, so can you!

You may want to build your own, or just use some elements, maybe you can suggest improvements and new ideas. Maybe you can design a different observatory dome for it? I have lots of further development ideas but not enough time to do them all! I would love to hear feedback if you have a go at something like this yourself.

Limitations

I have built and operated this in the UK in the Northern Hemisphere. Mechanically this will work in the Southern Hemisphere too, but I think that the software on the Raspberry Pi will need adjustments to work in the south. It's possible, but be prepared to dig into the software to iron out problems.

Image quality is NOT going to match expensive telescopes, this is an educational project to see how telescopes work. But it DOES work and can produce some really interesting images.

Safety

You are building a strong mechanical and electrical device that is to be used outside! Get advice from experts if you are unsure of anything here. This design is not weatherproof! Keep it indoors in bad weather and when not in use.

1. Protect fingers from the mechanical parts.
2. Protect power/electronics from the weather! At night, condensation forms and things get wet! The dome gives some protection from condensation, but not bad weather.

Assembly overview

Although this is mainly a 3D Printing project, I recommend that you start with the software and electronics first. Only start 3D printing the body when you are happy that the software and electronics work as expected. You can prove that the telescope will work electronically BEFORE you invest time and material in the 3D printing. If you do not decide to proceed, you still have the computer to use for other projects. You can practice using the telescope once you have the electronics working - it can simulate everything else.

Electronics

1. Build the Raspberry Pi
2. Build the Tiny2040
3. Test and focus the camera
4. Build the motorcontroller board
5. Test the motorcontroller board

Telescope

1. Build the foundation
2. Build the platform
3. Build the camera tower
4. Build the azimuth motor and gearbox
5. Build the altitude motor and gearbox
6. Install the electronics and camera
7. Build the observatory dome

Using Pi-lomar

1. Using the Pi-lomar software
2. Processing the images

Supplies

More details are given at each assembly stage, but here's an idea of the things you'll need. You can modify everything to your own preference if you want to.

Electronics

1. Raspberry Pi 4B. At least 2GB, preferably 4GB.
2. 32GB micro SD card.
3. Raspberry Pi USB power supply.
4. Pimoroni Tiny2040 microcontroller. 8MB Headered version.
5. DRV8825 stepper motor driver board (x2).
6. 0.9Degree Nema17 stepper motor (eg uk_stepperonline) **17HM19-2004S** (x2). (1A motors work fine, but I use 2A motors so that there's more power available in the future)
7. 12V 8A power supply.
8. Electronic components
9. Wire
10. Prototyping board
11. 47uF capacitors (x3)
12. 320 to 680 Ohm resistor
13. 10K Ohm resistor
14. BC548CTA NPN transistor (or equivalent)
15. 40way ribbon cable
16. 40way PCB header
17. Weatherproof protection for power and electronics.

Camera

1. Raspberry Pi Hi Quality camera sensor.
2. Raspberry Pi 16mm telephoto lens.
3. 30-40cm camera ribbon cable.

Mechanical

1. 6mm and 5mm diameter metal rod.
2. 1:60 worm gear set (x2).
3. M4 and M5 nuts and bolts 12-30mm.
4. M5 and M6 6x19mm skateboard/RC bearings.
5. M5 and M6 collars.
6. 250mm Lazy Susan bearing.
7. M6 flange couplers.

Computing/software

1. Raspbian OS (Latest BUSTER full build image).
2. Python 3.7 on Raspberry Pi.
3. Circuitpython 7.2 on Pimoroni Tiny2040 microcontroller.
4. Remote computer (whatever you like) to build, control and process images.
5. Download .zip file to install scripts, data and software onto Raspberry Pi and Tiny2040.
6. Software to 'stack' images. (eg DeepSkyStacker, or others)
7. Software to 'refine' images. (eg The GIMP, or others)
8. Internet access at least for the initial setup.
9. Local network (LAN cable or wifi) to operate the telescope remotely.
10. An SFTP client on your desktop computer. Filezilla works fine.

11. A terminal emulator such as puTTY or RealVNC on your desktop computer.

Printing

1. A 3D printer with a 300x300mm print bed. (Smaller is OK if you are prepared to modify some STL files).
2. Approximately 2kg of BLACK filament (PLA is enough). Use this for the interior parts to reduce reflections.
3. Approximately 2kg of WHITE filament (PLA is enough). Use this for the exterior observatory dome.

Time

1. Be prepared to spend some weeks building this. Hopefully the software building is quite fast, but there are a lot of components to 3D print and a few hours of electronics tinkering.

Skills

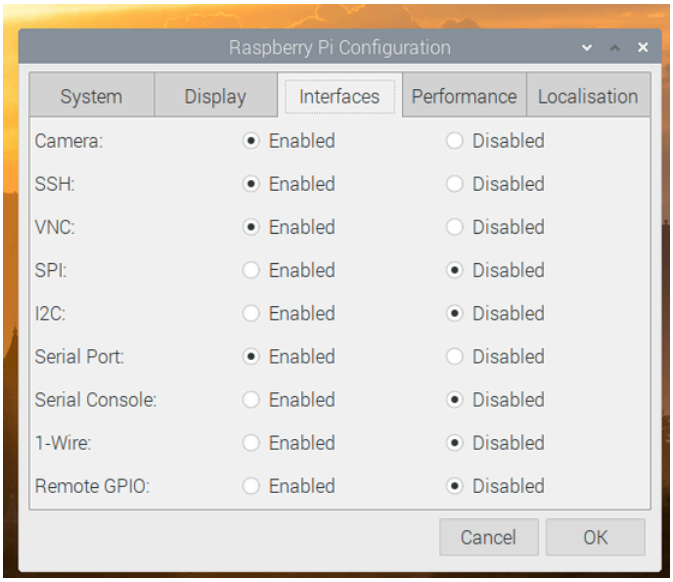
1. Basic 3D printing skills.
2. The ability to wire a simple circuit board for the Tiny2040 and DRV8825 chips.
3. An interest in astrophotography and astronomy. You will need to learn how to 'stack' images to get the best results from Pilomar. Astrophotography image stacking can be fun and creative.
4. Some basic Python.

Money

1. You will need to buy materials and components for this project. The most expensive single item is probably the Raspberry Pi, maybe you already have one of those!

Step 1: Build the Raspberry Pi





Pi-lomar works on a Raspberry Pi 4B with at least 2GB of memory. More memory helps. Use a 32GB micro SD card because Pi-lomar creates a lot of data. The Pi-lomar software uses several Python packages, most of them are already installed with a standard Raspberry Pi build, but it needs a few special items adding.

You need to follow the regular installation process for the operating system from the Raspberry Pi website. Pi-lomar does not yet work on the latest Bullseye OS, you must use the last **Buster** image instead. Install the full desktop version.

~~Operating system images – Raspberry Pi~~ ~~Raspberry Pi OS (Legacy) with desktop~~

- ~~Release date: September 22nd 2022 or later~~
- ~~System: 32-bit~~
- ~~Kernel version: 5.10~~
- ~~Debian version: 10 (buster)~~

23.Nov.2023: The originally specified OS image is no longer available via the Raspberry Pi imager. You can also access the Raspberry Pi archive to find a copy of many earlier OS images.

https://downloads.raspberrypi.org/raspios_armhf/images/

I recommend using the 2021-05-07 image as it is the latest Buster image available.

The Raspberry Pi website will guide you through the setup.

You then need to make some simple configuration changes using *raspi-config*. These enable remote control of the telescope and allow it to communicate with the Tiny2040 microcontroller.

- Set a suitable host name (eg pilomar)
- Enable SSH
- Enable CAMERA
- Enable VNC
- Enable SERIAL PORT
- Disable CONSOLE TO SERIAL PORT

Pi-lomar runs via a remote session ('headlessly') but it's a good idea to keep the RPi booting to the desktop (this seems to help auto-mount USB memory even though you won't use that desktop session often). Reboot the Raspberry Pi and log in.

The Pi-lomar software is available via GitHub here...

Short-bus/pilomar: RaspberryPi based miniature observatory (github.com)

You need to download the Pi-lomar software, directory structure and a script to install the software dependencies. GitHub lets you download the entire project as a zip file, simply extract that file into the /home/pi directory on your Raspberry Pi.

unzip pilomar-main.zip

mv pilomar-main pilomar

cd pilomar

cd src

chmod +x *

cd ../scripts

chmod +x *

You now have the basic directory structure under /home/pi/pilomar/, this includes all the initial files, programs and an installation script. If you have problems running the scripts, make sure that the files in the /src and /scripts directories have execute permission.

SECURITY

Before proceeding! Always check scripts that you download from the internet! Make sure you are happy that they are safe! The *scripts/buildpilomar* script will install several other packages that the telescope depends upon. At the command line enter the following commands

cd /home/pi/pilomar/scripts

./buildpilomar

Monitor the output, it will take several minutes to run. It will install a lot of dependencies and may need help if version conflicts appear in the future. Pi-lomar uses a stable version of Raspbian to minimize that risk!

Pi-lomar is now installed. Don't run it yet, it will complain that the microcontroller is missing.

Step 2: Build the Tiny2040



I used a Pimoroni Tiny2040 microcontroller. There are several other RP2040 based microcontrollers available, but I found the Pimoroni device is compact, reliable and easy to use. Use the 8MB headered version. These microcontrollers support MicroPython and CircuitPython, although they are similar this project's software is written for CircuitPython.

During development it is easier to use a breadboard to hold these components, you can test wiring more easily before you solder things into place permanently. This step is only installing CircuitPython and the motor control software.

Install circuit python on the Tiny2040 using these instructions. At this stage you only need a USB cable to connect the Raspberry Pi 4 and the Tiny2040 together.

The Thonny IDE that is included with the Raspbian O/S should be sufficient to read/edit/write/debug the code.py routine on the Tiny2040.

https://circuitpython.org/board/pimoroni_tiny2040/

I recommend installing this version of circuit python.

2022-02-24T18:08:08.000Z 1.4 MB adafruit-circuitpython-pimoroni_tiny2040-en_GB-7.2.0.uf2

When circuit python is installed, copy */home/pi/pilomar/tiny2040/circuitpython/code.py* from the Raspberry Pi onto the Tiny2040 and reboot it. The Tiny2040 LED will show PINK for a moment, then should occasionally flash BLUE. This indicates that the software is running and trying to communicate.

You can perform basic debugging by opening the SERIAL window in Thonny while the Tiny2040 is connected via USB to the RPi.

You can now remove the USB connection between the Raspberry Pi and the Tiny2040. You can now power the Tiny2040 by providing 3V3 power and a GND connection from the Raspberry Pi GPIO header. All further communication will be via the UART pins in the GPIO header.

*WARNING: BEWARE having the GPIO HEADER**AND** the USB cable connected to the motorcontroller at the same time. It may provide two conflicting voltages to the microcontroller. It may damage something.*

The Pimoroni Tiny 2040

You can use any 3.3V microcontroller that supports CircuitPython. If you want to use MicroPython you will have to make some changes to code.py because it's not a 1:1 equivalent, but the changes are relatively minor. If you want to use a different microcontroller board, you'll probably also need to change code.py and the pinouts in the circuit schematic later in this project.

I found Pimoroni's mighty Tiny2040 works better than the alternatives I tried, I recommend it.The designs here are specific to the Tiny2040.

Step 3: Test and Focus the Hi Quality Camera





Install the Hi Quality camera module on the Raspberry Pi. The official documentation gives good instructions.

Raspberry Pi Documentation - Camera

You do not need to install any additional libcamera libraries, Pi-lomar currently works with the earlier camera libraries included in the Raspbian Buster build you have just installed.

I recommend using about a 30cm ribbon cable for the camera connection, it makes life easier.

The Pi-lomar software is configured for the 16mm 'telephoto' lens that was originally recommended for the Hi Quality camera. This gives a field of view of about 21degrees of the sky. (You can use different lenses, but will have to make some parameter changes.)

The final step is the focusing of the telescope lens. Luckily everything in the sky is far away, so we just need to set the focus to infinity and leave it there.

On a bright day point the camera at a very distant object and adjust the focus until it is crisp. Alternatively on a moonlit night, point it at the moon and focus on that.

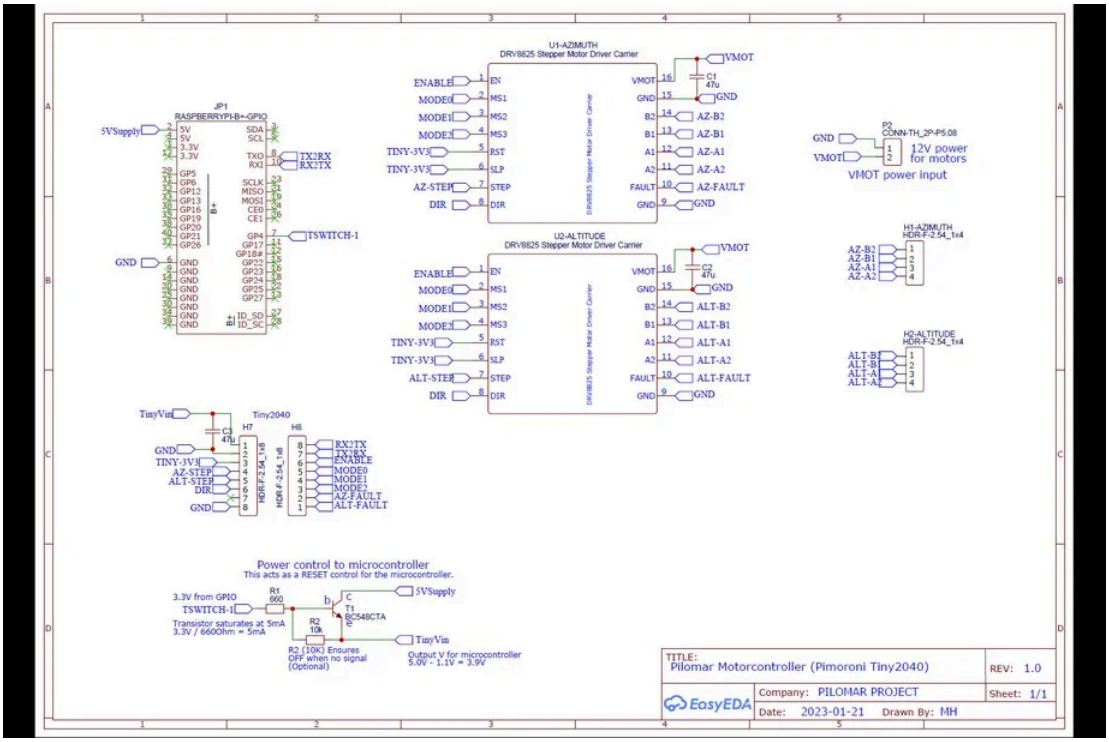
The raspistill program has a useful focusing tool. Open a command line window from the GUI desktop and type the following command

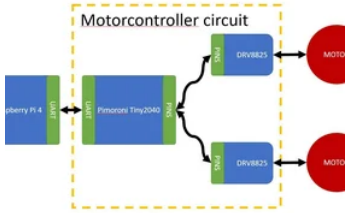
raspistill --focus -o test.jpg -t 60000

You will see a live image from the camera for 60 seconds, as you adjust the focus you will see a score on the image telling how sharp the focus is. Adjust focus on the lens until the image is sharpest and the score is highest. When you are happy that objects a very long way away are in focus tighten the focus ring screw. You shouldn't need to change it again.

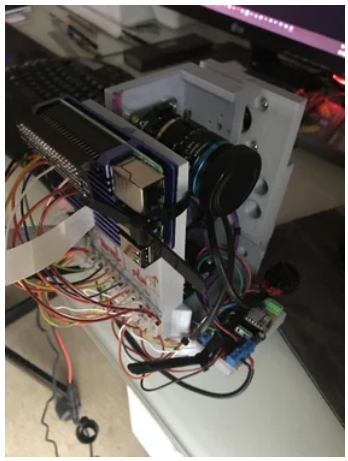
I also set the aperture of the lens to about f2.8 or a little smaller as this increases the depth of field for the lens, making focusing easier. Don't make the aperture TOO small, the images will be too dark!

Step 4: Build the Motorcontroller Board









The motorcontroller board is the interface between the high level functions of the Raspberry Pi and the physical control of the telescope position. This might feel like the most frightening part, but take your time and enjoy the journey! The telescope has successfully operated with several different motor control circuit boards. The first one was very basic, just a mass of jumper wires and loose components.

I include a schematic here, you can construct the final circuit board to your own preference. The circuit board supports the Pimoroni Tiny2040 microcontroller and the DRV8825 stepper motor chips. The level of sophistication is up to you. You can hack your own circuit board, or if you are comfortable with PCB designers you can even design a board and have it produced for you. My current board was fabricated by the online supplier PCBWay at a reasonable price.

The concept of the circuit is to make a simple motor controller 'hat' for the Raspberry Pi. This means connecting power and UART pins from the RPi GPIO header to pins on the Tiny2040 microcontroller. Then connecting Tiny2040 output pins to the two DRV8825 driver chips. Finally after connecting a 12V supply and motors to the two DRV8825 chips you have a working motorcontroller. The Tiny2040 lacks a 'reset' pin, so the circuit includes

a transistor which controls the power to the Tiny2040. When we turn the transistor ON the Tiny2040 is powered, when the transistor is OFF, the Tiny2040 is turned off - acting as a simple reset.

It is wise to start by using breadboard and some 'jumper jerky' to get things running initially. Transfer to something more permanent when you are happy everything works as expected.

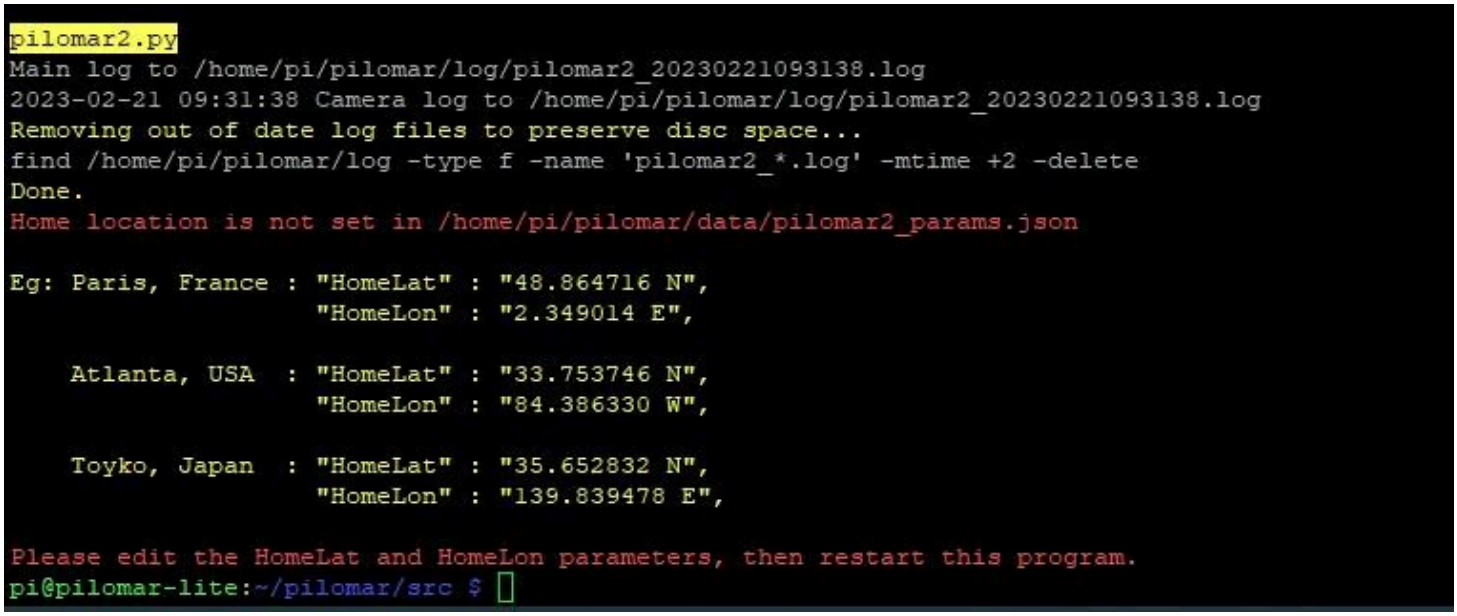
I suggest that you build the Tiny2040 element of the circuit first, this will let you run the Pi-lomar software. You can add the connections to the DRV8825 stepper drivers and motors after that.

When you do add the DRV8825 drivers into the circuit be sure to set the current limiting control to the correct value to protect your motors. I have used 2A motors, but I limit the current to about 0.7A, you can increase the power later if you find you need to. There are excellent online guides to setting the current limit, it is a simple process.

Be sure to remove the USB cable from the Tiny2040 before connecting to the Raspberry Pi GPIO pins. You may cause power problems if both are connected at the same time.

Now when you boot the Raspberry Pi, it will also boot the Tiny2040. You can now test the software.

Step 5: Test the Motorcontroller Board



Open a command line window or puTTY terminal connection to the Raspberry Pi as the pi user.

cd pilomar/src

./pilomar

This runs the Pi-lomar program, the software will initialize and create a default parameter file. It will quickly exit asking for the home location to be configured in the new parameter file. It will show some examples of locations and how to code them. Use an editor such as nano to modify the listed json file and add your home latitude and longitude. If unsure, ask a web search engine for the latitude and longitude of where you live.

cd ../data

nano pilomar_params.json

Edit the "HomeLat" and "HomeLon" values.

<ctrl>-o to save the new values.

<ctrl>-x to quit the editor.

You can now restart the Pi-lomar software.

cd ../src

./pilomar

Choose a target. I recommend a solar system target, choose one that is above the horizon. It doesn't matter if it's daytime, we're just testing the motor, not taking photos yet.

When the menu appears, select the 'Motor Tools' menu, then the 'Tune Azimuth' option. When prompted enter 1000 and press <enter>. After a few moments you should see the Tiny2040 LED turn green for a moment, then return to the prompt. This proves that communication is working. If there is power to the motors the azimuth motor should move too. enter 'x' to exit back to the menu.

Use the TUNE ALTITUDE option to set the correct starting position for the camera cradle. When the camera is mounted on the cradle, it should be pointing horizontally, looking at the horizon, this is 0 degrees altitude. The 'home' position of the camera. If the telescope ever looses its position, use these two TUNE options to recover the situation.

If the Raspberry Pi is not communicating with the microcontroller, it will terminate with an error. In which case, go back and check the circuit and previous steps. It is normal to see a message that the motorcontroller reports a restart.

From this point onwards, you can use the Pi-lomar software even without the rest of the telescope! It will happily try to work with just the Raspberry Pi and microcontroller connected.

You can even disable the camera from Pi-lomar's camera menu, in which case the telescope will simulate the images instead. You can practice and develop the software further this way.

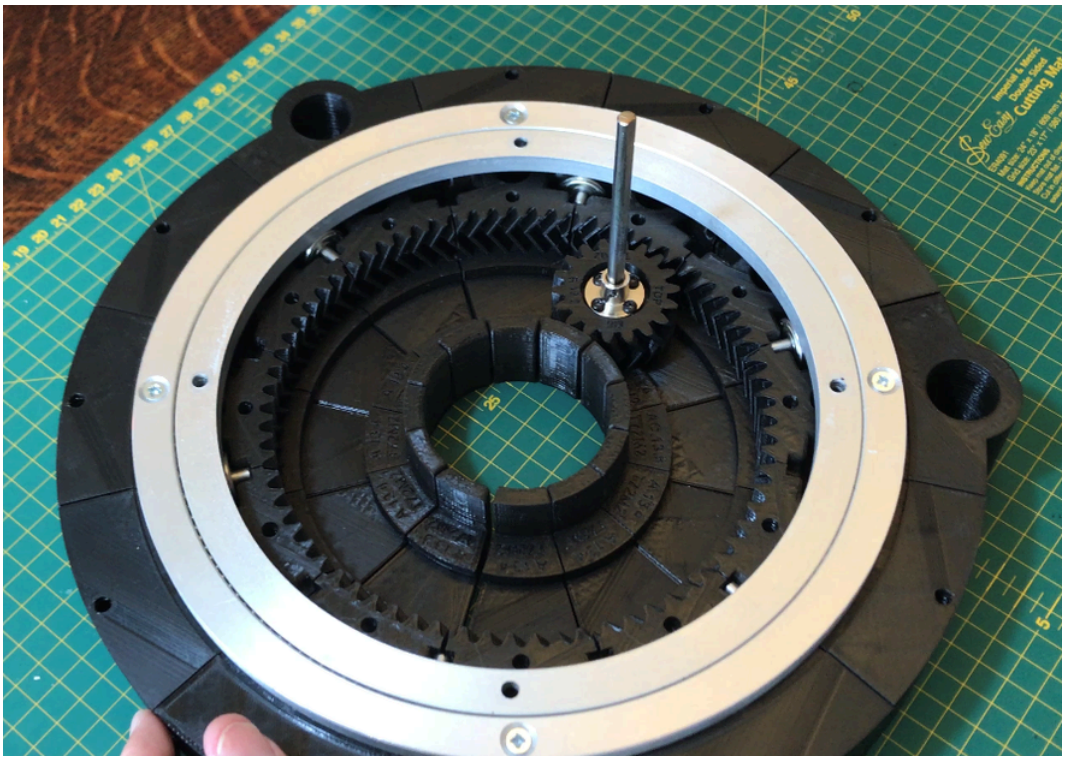
puTTY configuration

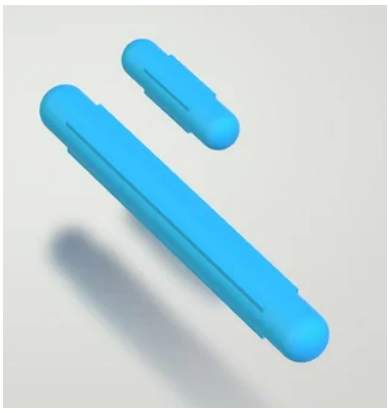
The Pi-lomar software runs under Python3.7, which is UTF-8 compliant. Make sure that your puTTY configuration is set for UTF-8 characters too, you can get some weird character corruption otherwise.

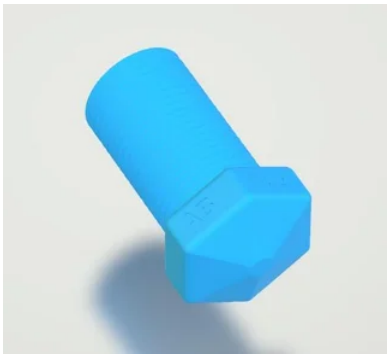
RealVNC connection

Instead of puTTY, you can also use a virtual terminal application such as RealVNC, it's more robust if you have wifi connectivity problems, but more fiddly to set-up sometimes. There are good guides online for this, and the Raspberry Pi comes with some elements already installed.

Step 6: Build the Foundation







Now we get to the 3D Printing part of the telescope. The telescope consists of 5 major assemblies. The Foundation, the Platform, the Tower, the Gearboxes and the Dome. Take your time with printing the components, I estimate you need about 4 spools of filament (1.75mm filament, 1kg per spool). 2 spools of Black and 2 spools of White are ideal, but why not get creative?

3D Printer settings

- The telescope prints nicely with PLA material.
- I recommend around 0.12mm layers for a few small fine-detailed components.
- I recommend around 0.2mm layers for the majority of components.
- Components are generally stronger if you have 3-4 layers for wall, floor and ceiling thicknesses and 20-40% infill.
- If your prints are reasonable quality, no further strengthening is needed. However if you have issues with layer separation, some of the more critical parts have holes in them which you can use to add supporting bolts or just fill with adhesive to strengthen them.

The foundation is the static base that the telescope sits on. It consists of a three footed base which supports a Lazy Susan bearing, this bearing takes the weight of the upper telescope body and allows it to move left-right (azimuth).

The .STL files for the foundation components are available here.

There are a couple of brief videos to take you through the assembly.

Make sure that you purchase the 250mm Lazy Susan bearing. They come in many different sizes! The bearing usually comes with 4 small white plastic feet attached, these pull off - you don't need them. Although most of the telescope uses M5 nuts and bolts, please use M4 bolts with recessed heads to attach the bearing to the foundation. This ensures that the bolt heads sit beneath the surface of the bearing, so it will not catch on the body of the telescope as it moves.

The first component built is the drive wheel. There are two versions, one for an M5 shaft and one for an M6 shaft. Use a shaft diameter that matches the bore of the Worm Gear wheel. You can buy those in M5 and M6 variants.

The little support bearings that sit beneath the Lazy Susan bearing can be budget bearings for radio controlled cars etc. They are 5*19*6mm bearings. 5mm inner diameter. Use small pieces of M5 rod or just M5 bolts as the axles for these bearings then insert them into the foundation, evenly spaced around the circumference. 6 of these bearings are enough.

Step 7: Build the Platform

The platform sits on top of the Lazy Susan bearing and rotates the telescope and dome, it holds the azimuth motor, gears, camera tower and the outer observatory dome. I printed this in BLACK PLA.

This platform bolts to the inner ring of the Lazy Susan bearing. Again using M4 recessed bolts to avoid snagging.

Use the DaM4 nut holders to locate the nuts for the M4 bolts in the platform.

Pay attention to the single DaM5 nut holder which needs to be inserted into the platform base during assembly. See the video. DaM4 and DaM5 create captive nuts to help with assembly.

If you are happy with the assembly you can use a little silicone sealant when you insert the pegs into the platform segments, this helps to reduce noise later on but remains flexible.

The .STL files for the platform components are available here.

There is a brief video to take you through the assembly.

Step 8: Build the Camera Tower

The camera tower sits on the platform, it holds the electronics, altitude motor, gears and camera. I printed this in BLACK PLA.

The camera tower consists of two inner walls which are joined by a camera cradle. The camera cradle is mounted on bearings so that it can rotate freely up and down. The camera cradle is moved by a stepper motor and gearbox.

Each inner wall is made of two wall pieces which are bolted together to hold the RC bearings which support the camera cradle. See the video for more information.

There are two versions of the inner wall available. P.14.0 is a lower profile which gives a little more space under the dome, P.13.6 has more bolt holes available to help mounting electronics to the tower. The video uses P.13.6.

The .STL files for the camera tower are available here.

There is a brief video to take you through the assembly.

Step 9: Build the Azimuth Motor and Gearbox

The azimuth motor and gearbox sit on the platform and control the motion of the telescope left and right (azimuth). I printed this in BLACK PLA. The motor sits on the observatory platform and drives the worm gear through to the azimuth drive wheel. The motor has 400 steps per revolution, the worm gear multiplies this by 60:1 to give 24000 steps per revolution, the final azimuth drive wheel multiplies by a further 4:1 giving 96000 steps per revolution. This means Pilomar can locate the telescope to 0.004 degree precision.

The gearbox is designed to let you align all the components with some flexibility. You should be able to adjust and finetune the positions and contacts between all the components to achieve a smooth and reliable drive.

The .STL files for the motor holder and gearbox are here.

There is a brief video to take you through the assembly.

NEMA17 motor notes

Both gearboxes use the same stepper motor type. This is a NEMA17 stepper, with 0.9Degrees per step (400 steps per revolution). You can use a 1A motor, but I chose 12V 2A motors because it offers more flexibility, doesn't stress anything, and you may want that extra power in the future if you want to use 'microstepping' to increase precision. With microstepping you can achieve 0.45Degrees per step easily (800 steps per revolution of the motor shaft) - but will loose some motor power in exchange. The basic design does not use microstepping, but the motor controller circuit and software will support it.

The stepper motor has an M5 shaft, ideally 'D' shaped so that the worm gear can be firmly connected to it.

You need the 4 wire bipolar stepper motor to match the DRV8825 chips, do NOT use the 6 wire unipolar version. I used uk_stepperonline's model: **17HM19-2004S***0.9deg Nema 17 Stepper Motor 46Ncm(65.1oz.in) 2A 12V 48mm 4-wire CNC 3D Printer*

The Nema motors are 'current' limited rather than voltage, which means the most important thing to control is the current through the motor. There are discussions online about voltage tolerance for stepper motors. If you can only get lower *voltage* motors, they may still work, but check online first. The current through the motor is controlled by the DRV8825 driver chip. There are good explanations online about how to adjust the current limit - you need a multimeter and a screwdriver to adjust the voltage on the little potentiometer.

Step 10: Build the Altitude Motor and Gearbox

The altitude motor and gearbox are attached to the camera tower, they control the motion of the telescope up and down (altitude). It is similar to the azimuth gearbox, but has some additional components. I printed this in BLACK PLA.

The .STL files for the motor holder and gearbox are here.

There is a brief video to take you through the assembly.

Step 11: Install the Electronics and Camera

The electronics and camera were prepared earlier, but now need to be fitted into the observatory. I printed a couple of simple mounting plates in BLACK PLA. Your design may change depending how you build the circuit boards themselves. The mounting plate for the Raspberry Pi 4 should work for you though.

Mount the camera and lens (already focused on infinity) in the camera cradle with 4 2.5mm nylon bolts.

Attach the Raspberry Pi and Motorcontroller board to the side of the camera tower. There are free M5 bolt holes available which should help you.

Connect the motor wires to the motorcontroller board. 4 wires per motor. Make sure the wires are connected in the correct sequence for your particular motor.

The 12V power cable for the motors and the 5V USB power cable for the Raspberry Pi should be passed through the hole in the centre of the foundation of the telescope. This will prevent them getting twisted as the telescope turns. You can secure them with cable ties to the camera tower just above the opening.

The 12V power supply could be quite powerful, at maximum you can deliver 2A per coil per motor! 2 coils per motor, means 8A in total! Don't damage those coloured motor wires!

Related .STL files are here.

See the photographs for an idea of how the boards can be attached and cables routed. There are several spare bolt holes on the camera tower which can be used to wires with cable ties. Be sure to keep the wires away from the gear teeth!

After this step, the telescope should be fully operational.

Revisit the instructions for **Test The Motorcontroller Board** to position the telescope at it's home location.

- Use the HOME MOTOR option on the menu to move the motors to their home position.
- Use the TUNE MOTOR option on the motor tools menu to adjust the physical position of the camera to be horizontal.

Step 12: Build the Observatory Dome

The telescope is already operational, but it still needs some environmental protection. This build of Pi-lomar is not weatherproof, but by adding an observatory dome you make it easier to use in cold temperatures for real observations. Never use it in bad weather, and keep it inside when you are not using it.

You are free to create your own dome design.

I printed this in WHITE PLA.

The .STL files for the Pilomar dome are here.

There is a brief video to take you through the assembly.

Step 13: Using the Pilomar Software

It is common with astrophotography to take a large batch of photographs of the target, then combine these in a process known as 'stacking'. Pi-lomar does not (yet) perform the stacking, but it collects the batch of photographs for you.

You can take a single photograph with Pi-lomar, but you will get a much better result if you take a batch of them then stack them. Stacking can improve image quality dramatically, and if you add some other control images ('darks','biases' etc) you can reduce noise to really pull out detail from your photographs. Pi-lomar will collect all these images for you ready for stacking.

Setting up the telescope

Place the telescope outside on a flat surface, ensure that all the power and network connections are in place and safely protected from the weather. Point the body of the telescope due South with the camera horizontal, pointing at the horizon. This is the HOME POSITION. You can use the 'Tune Azimuth' and 'Tune Altitude' options on the 'Motor Tools' menu to get this initial position set up.

Starting the observation

From your desktop computer (hopefully somewhere warm and comfortable) open a remote terminal connection via puTTY to the telescope. The display seems to work best on a puTTY connection. Then open the Pilomar software directory and start it up.

cd pilomar/src

./pilomar

The software may download some data from the internet the first time it runs, it's loading catalogues of targets from public sources. If it doesn't have your home location configured, it will ask you to update the parameter file and restart the software.

You will be asked to select a target. Choose the type of target (solar system, messier, NGC, Hipparcos etc). Then enter the name of the target you want to select. If the target is not visible you will have to select something else. This selects the Orion Nebula. A great first target!

Select target > Messier objects

Target name: M42

You will return to the main menu. Choose 'Begin observation'.

The telescope will calculate the location and trajectory of the target and pass this to the motorcontroller. When the motorcontroller has the configuration and trajectory the telescope will move into position and start taking photographs. It can take a minute or so for the telescope to get everything ready.

The telescope will then begin to track the target and take a series of 10second photographs. You can let it capture 100 images, or stop it at any time by pressing 'x'.

When the set of photographs has been collected it will return to the main menu. You can take more photographs, select a new target or adjust the settings. Maybe you want longer exposures? Maybe you want a larger batch of images collected?

The photographs are called 'light' images and are stored in a specific folder for your target. You can view/copy these photographs from the GUI desktop easily. Or use something like Filezilla to transfer them onto another computer for further processing.

There are lots of options for taking images, so feel free to try different options on the menus and see what happens.

If you want to photograph the same object several nights in a row, Pi-lomar remembers the recent targets and settings, you can resume those from the 'Repeat earlier observations' option when selecting a target.

Step 14: Processing the Images

So you have a bunch of photographs captured, individually they don't look particularly special. You could do the same with the camera on a tripod.

The next task is to 'stack' these images, combining them intelligently into a single image. Using the data from all the images to produce a single cleaner image.

When an observation completes, Pi-lomar tells you where the images are stored. On your desktop computer use a file transfer program such as Filezilla to download these folders onto your desktop.

The 'light' folder contains traditional '.jpg' image files - which are easy to look at. It also contains raw image files in the '.dng' format. These contain more pure information taken directly from the camera sensor. Astrophotography software can use these raw image files directly to produce enhanced images.

I use multiple free astrophotography stacking programs, a relatively simple one to use is DeepSkyStacker. The help option on DeepSkyStacker's menu is a good place to start. There are excellent tutorials on YouTube too, but lets make a REALLY simple image here.

Select *Open picture files...*

Choose all the '.dng' images in the 'light' folder.

Select *Check all*

Select *Register checked pictures...*

Ignore the warning about adding dark, flat and offset files. You can take those too with Pilomar and introduce them into the stacking process as you get more practice.

Make sure that the *Stack after registering* checkbox is ticked.

Select OK on the *Stacking Steps* window.

After a few minutes you will have an initial stacked image. You can now tweak the settings.

Adjust the RGB/K Levels sliders so that all three colour peaks are on top of each other. Press APPLY to see the result.

On the Luminance panel, adjust the 2nd Midtone slider so that the black line Just starts to climb from zero at the left side the three colour peaks. Then press APPLY. (Play with this slider to see it's effect).

On the Saturation panel you can increase saturation to around 20% or 30% to exaggerate the colours more. Press APPLY again.

Keep playing with these sliders until you see a result you like.

On the main menu select *Save picture to file...*

In *Options*, Select *Apply adjustments to the saved image*

Choose a location and name, Deep Sky Stacker saves images as TIFF files. You should still be able to open them with most image applications.

You have stacked your first image. You can process the image further in an image editor such as The GIMP. That will let you adjust colours, crop, play with light levels etc.

It's great fun to play with all the options for stacking and adjusting the images to get the most detail from them. There is LOTS to learn about image processing, enjoy the journey!

Step 15: Summary

This project demonstrates one route to making an automated telescope, it is on a small scale, but the principles work even if you want to produce a larger telescope.

There are many enhancements possible for this design, and many features that could be added.

- Make it battery powered and portable?
- Add GPS and sensors to make setup more automatic?
- Automate the image stacking into the capture process?
- Design different domes?
- Use different gearing for even finer movement control?
- Support different cameras and lenses?
- Maybe even a smart-phone cradle?
- Very strong telephoto lenses may overwhelm the current tracking system.
- Conventional SLR lenses will produce higher quality images.
- Develop a web front end instead of the character interface?
- Build smarter star tracking or 'plate solving'?
- Convert from python/circuitpython into a faster language?
- 3D Print the entire thing at twice the scale and see how big you can go?

The software is still evolving because I use this regularly, it would always benefit from some cleaning and simplification, it's an experimental environment in practice.

You don't have to build the 3D printed telescope, you may want to make yours from other materials and methods, the software and electronics will adapt to fit.

It would be great to see if anyone finds a use for any elements of this project. I'm likely to use this as a starting point for a larger telescope now... see you in a few years when that's done!